Idle Cycles Are Not Wasted Cycles

Matheus Stolet
mstolet@mpi-sws.org
Max Planck Institute for Software Systems
Germany

Simon Peter
simpeter@cs.washington.edu
University of Washington
USA

Antoine Kaufmann
antoinek@mpi-sws.org
Max Planck Institute for Software Systems
Germany

Abstract

Idle network service cores are treated as wasted compute. This assumption motivates increasingly sophisticated mechanisms that reclaim idle cores at microsecond timescales. We argue that this view no longer matches modern server hardware. On contemporary multicore processors, active cores compete for a shared package-level power and thermal budget. Once that budget becomes the limiting resource, an idle core that waits efficiently returns compute capacity that hardware can redistribute to productive work. Measurements on a recent AMD EPYC processor show how waiting strategy, processor topology, and idle duration determine this tradeoff. Our results suggest that reclaiming idle cores often yields less benefit than commonly assumed while introducing substantial scheduling complexity. We propose a budget-centric view of service-core systems in which power, rather than core occupancy, becomes the fundamental resource and waiting policy becomes a first-class systems design choice.

CCS Concepts

• **Software and its engineering → Operating systems**; • **Computer systems organization → Multicore architectures**.

Keywords

service cores, network stacks, multicore processors, power management, chiplets, waiting policies

ACM Reference Format:
Matheus Stolet, Simon Peter, and Antoine Kaufmann. 2026. Idle Cycles Are Not Wasted Cycles. In *The 25th ACM Workshop on Hot Topics in Networks (HotNets '26), November 16–17, 2026, Salt Lake City, UT, USA*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3847290.3859941



This work is licensed under a Creative Commons Attribution 4.0 International License.
HotNets '26, Salt Lake City, UT, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-3029-0/2026/11
https://doi.org/10.1145/3847290.3859941

1 Introduction

High-performance network stacks increasingly use *service-core architectures*: packet processing runs on dedicated cores as a shared service, and applications communicate via shared-memory queues rather than system calls [20, 27, 31, 34, 35]. Dedicating cores keeps applications and the stack from competing for caches, branch predictors, and other core-local state; parallelises processing; removes context switches and privilege transitions from the data path; and lets one shared instance multiplex many applications.

But dedicated service cores are rarely busy every cycle. Load fluctuates at microsecond timescales, and demand does not align with integer multiples of a core. The community treats these idle cycles as waste to be eliminated, and has built sophisticated machinery to do so, such as core reallocation at microsecond granularity, dedicated scheduler cores, and elaborate handoff protocols [10, 27, 32]. This machinery follows a seemingly obvious mental model: a core-cycle spent idle is a core-cycle of computation lost. We argue that this model is wrong on modern servers, leading to systems more complex and *less* efficient than doing nothing.

Large multicore processors face fixed power and thermal budgets [7, 11]. For instance, an AMD EPYC 9655 guarantees 2.6 GHz across its 96 cores but boosts a single core to 4.5 GHz. This gap shows that N cores do not deliver N cores' worth of peak compute. Production fleets oversubscribe power, so saturated machines routinely run against their caps [9, 21, 23, 38]. On our EPYC 9655, a cache-resident parallel workload running on all 96 cores achieves only 47% of the per-core throughput it achieves alone. Thus cores are not independent units of compute: performance depends on a shared budget.

Below saturation, reclaiming a service core offers little. If an application is not fully utilising its current cores, reallocation cannot improve performance. If it is saturating them, the scenario that motivates microsecond core reallocation, the processor is constrained by its power budget. Elastic allocation therefore only helps workloads for which this underlying accounting no longer holds. In that regime, the arithmetic of idleness inverts and an idle core does not squander its capacity. Instead the processor's firmware redistributes it in microseconds with no software in the loop.

We quantitatively characterise this regime and measure how waiting strategy, idle patterns, placement, and reallocation determine how much budget idle service cores return. Our insights lead us to propose a replacement mental model: **the processor is a fixed compute budget, not a collection of cores.** Below the power cap, cores behave classically; above it, they compete for a shared budget, and one must consider how much budget they consume instead of how many cores a service holds. How a core *waits* becomes as important as whether it is allocated: hardware waiting mechanisms—`PAUSE`, user-level monitor/wait, halt-class deep idle—return progressively more budget at higher wakeup latency without giving up the core. Blocking, by contrast, releases the core to a scheduler, paying wakeups, cache refills, and engineering complexity to redistribute a budget the hardware already redistributes for free. This cost is worthwhile only once idle periods amortise it; power capping can shift this break-even point toward longer idle periods.

We distil four lessons for service-core architectures and related multicore systems. (1) Waiting returns power and thermal headroom, so idleness is a transfer. (2) The compute budget is distributed hierarchically across the CPU, so task placement decides who shares it. (3) Waiting trades wakeup latency for returned budget, so waiting policies should adapt to the workload. (4) Core reallocation is worthwhile only after blocking overheads are amortised, so our systems should stop paying the cost of complexity to eliminate idle cycles that were never wasted to begin with.

2 Background

Modern network stacks and processors have evolved considerably over the past decade, changing the trade-offs around dedicated polling. Many of the assumptions that shaped earlier network stack designs no longer hold on modern servers.

2.1 Managing Service Cores

Service cores isolate operating system services on dedicated cores to reduce interference with applications. This separation helps latency-sensitive systems but complicates allocation. Service demand varies over time, yet cores are often provisioned for peak load to maintain low latency during bursts. This leaves reserved capacity idle when demand dips.

Microsecond core reallocation. Recent work treats idle service cores as stranded resources that should be reclaimed. These systems dynamically reallocate cores between applications and services as demand changes, often at microsecond timescales, to recover otherwise idle polling cycles [10, 28, 32]. These mechanisms incur direct coordination and migration costs and indirect costs from disrupting caches and other microarchitectural state when a core runs different work. Both must be amortised over the reclaimed work.

Low-overhead core handoffs. A complementary line of work reduces the latency of handing a core new work. These systems employ low-overhead preemption, yielding, and interrupt delivery to shorten the critical path between an idle core and useful execution [2, 16, 17, 24]. These mechanisms reduce the cost of scheduling decisions, but they do not assess if reclaiming idle service cores is beneficial.

Waiting strategies. Service cores use different waiting strategies for short-term idling. Busy polling does not reduce core power consumption but resumes immediately. `PAUSE` spin-loop hints let the core reduce power and, with multithreading, enable the peer thread to use more cycles, while preserving 10–100 ns-scale wakeups [15, 22]. `UMWAIT` and `MWAIT` save more power by entering deeper idle states [15] but with increased wakeup latency. Deeper power-saving states, such as `HLT`, require slow, interrupt-driven wakeups. These strategies form a spectrum, trading energy and multithreading efficiency against wakeup latency.

2.2 Power and Thermal Management in Large Multicore Machines

The large multicore processors on which these services run operate within a package-level power limit. Hardware and firmware continuously allocate this finite limit across active cores, reducing their frequency when concurrent activity exceeds the package's power or thermal envelope. Thus, activating another core reduces the frequency and the performance of work running elsewhere on the socket [7, 11].

Processor performance depends on package activity. We use the AMD EPYC 9655 to illustrate this constraint. The processor integrates 96 cores within a 400 W default TDP and advertises a maximum single-core boost frequency of 4.5 GHz, but an all-core boost frequency of 4.1 GHz [1]. The gap between maximum single-core and all-core frequency exemplifies that the frequency available to each core depends on activity across the package. Consequently, cores devoted to polling can consume power headroom even when they perform no useful network processing.

Topology influences power and thermal management. The EPYC 9655 organises its cores into twelve eight-core compute dies. Because each compute die is physically distinct, its power density, temperature, and available frequency differ as load is distributed across the package [26, 33]. A polling thread may therefore interfere differently with application threads depending on whether they share a core, a die, or the socket. Task placement is consequently a power-management and cache-locality decision. Figure 1 illustrates power and thermal management in a modern chiplet CPU.

Power constraints have tightened since Dennard scaling ended and are likely to intensify as core counts grow faster than power efficiency improves [7, 11].

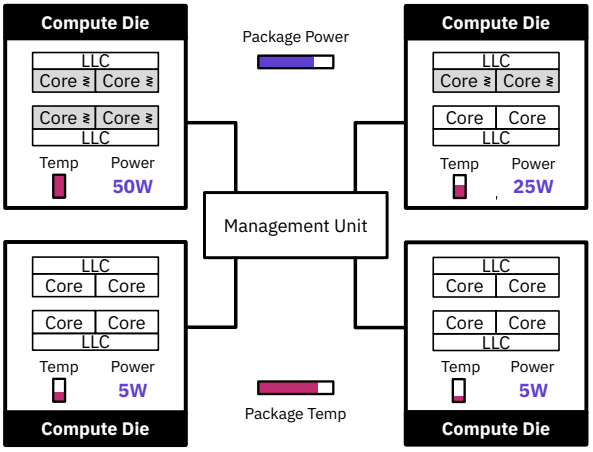


Figure 1: Chiplet-based CPU with four compute dies and four cores per die. The management unit controls power and frequency across cores by monitoring temperature, voltage, and performance counters in individual chiplets and keeps the package under its limits.

3 The Fixed Compute Budget Model

The conventional view of multicore processors treats each core as an independent unit of compute. Under this model, an idle polling thread wastes compute because its cycles could instead execute application instructions. This intuition holds while the processor has unused power and thermal headroom. In this section, we evaluate application performance under constrained thermal and power regimes with systematic microbenchmarks and show that this intuition breaks when the processor reaches its package limits. From these results, we propose lessons for service-core network stacks (L1–L4) and a new mental model where the processor behaves like a unit under a fixed compute budget, rather than a collection of independent cores. Finally, we use these insights to demonstrate improved end-to-end RPC throughput with a real service-core network stack.

3.1 Experimental Setup

The microbenchmarks are run on a machine with Linux kernel version 6.12.86 and a single-socket AMD EPYC 9655 processor with 12 dies, 8 physical cores per die, 96 physical cores in total, and 192 threads with SMT enabled. We sample package power with Linux RAPL counters and per-workload CPU frequency with per-core frequency counters.

Benchmarks. Across experiments, we use a matrix multiplication benchmark as the compute baseline and several polling microbenchmarks as waiting baselines. The matrix multiplication workload operates on 16×16 matrices with either scalar or 512-bit vector AVX instructions. Matrices fit in the L1 cache to avoid cache and memory contention. The polling baseline runs in different modes that represent different waiting strategies (`Busy`, `Pause`, `Block`) and pseudo-packet processing tasks (`Work`). `Busy` repeatedly spins on a shared value, `Pause` performs the same spin loop but inserts the `PAUSE` instruction in the poll loop, and `Block` sleeps between polls. `Work` combines polling with simulated IP/TCP header accesses and checksum computation. We pin workloads to specific cores and average results over three 60-second runs, unless otherwise stated.

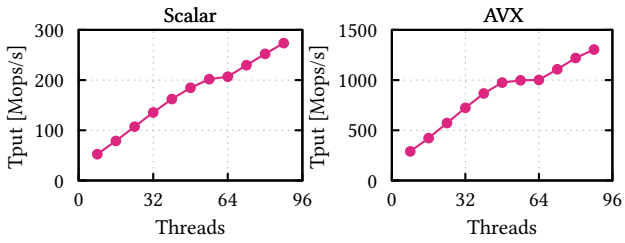


Figure 2: Aggregate compute throughput as the workload increases the number of cores. Throughput briefly flattens near the package's power and thermal knee, after which each additional core contributes less work.

3.2 The Power and Thermal Knee

Core count is a poor proxy for compute capacity once a processor approaches its package limits. To expose this effect, we run a matrix multiplication workload on an increasing number of physical cores. Figure 2 reports the throughput for scalar and AVX baselines. Initially, each additional core contributes substantial useful work. The curve then reaches a knee in which throughput is nearly flat while more cores are activated, before increasing again at a lower rate.

Package-level power and thermal limits reduce per-core performance. Activating another core increases potential parallelism, but also forces the processor to divide finite electrical and thermal headroom. Dynamic voltage and frequency scaling responds by reducing core frequency. Around the knee, where the processor starts to more aggressively downclock, the loss on already-active cores can nearly cancel the work provided by a newly activated core. Beyond it, throughput still grows, but the incremental contribution of each core is smaller. Before the knee, each additional core contributes an average of 4.24 million operations per second for scalar matrix multiplication. Beyond the knee plateau, that falls to 2.79 million operations per second. For AVX, the contribution drops from 22.6 to 12.6 million operations per second per additional core.

SMT threads provide little additional compute. Figure 3 shows that once the processor exhausts its physical

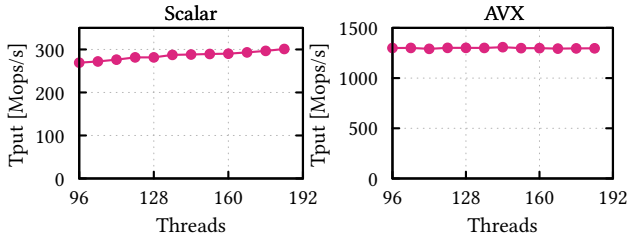


Figure 3: Aggregate compute throughput flattens once the workload scales beyond physical cores to SMT.

cores, SMT threads contribute even less additional compute. For compute-intensive workloads, such as AVX, aggregate throughput nearly plateaus despite activating more hardware threads, while scalar workloads see minimal throughput increase. Here adding cores provides diminishing returns because they compete for the same package budget.

Fixed-Budget Model. Below the processor package limit, additional active cores increase compute. Above this limit, additional cores yield marginal performance gains. Activity on one core reduces the budget available to others. Idling offers that budget back for redistribution.

3.3 The Waiting Spectrum

In the fixed-budget regime, a thread that enters hardware idle states returns compute budget that the processor can redistribute to productive work. We isolate this redistribution with an eight-thread matrix multiplication foreground workload. We then place an increasing number of background threads on otherwise unused physical cores. The background either performs matrix multiplication or polls queues, akin to a low-latency network stack, with different waiting strategies. Figure 4 reports foreground throughput, while Figure 5 reports core frequency and total power.

Waiting strategy determines foreground application performance. Scalar compute-heavy background work reduces foreground throughput from 26.8 to 20.5 Mops/s, and busy polling reduces it to 22.5 Mops/s. In both cases, package activity consumes headroom and the processor downclocks the foreground. By contrast, `Pause` polling retains foreground throughput and frequency while using much less power than `Busy`, leaving headroom for the foreground.

AVX workloads magnify the cost of budget contention. AVX instructions amplify the effects of the fixed budget as the chip reaches its limits. Throughput drops from 155.9 to 130.9 Mops/s with background busy polling and falls further to 98.9 Mops/s with vectorised matrix multiplication. `Pause` polling again preserves foreground throughput with the same eight-core allocation as more cores are added.

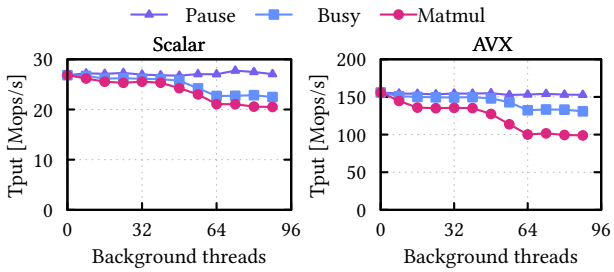


Figure 4: Foreground matrix multiplication throughput as increasing core counts perform background work (`Matmul`, `Busy`, or `Pause`). `Pause` preserves foreground capacity, whereas compute and busy polling consume the shared package budget.

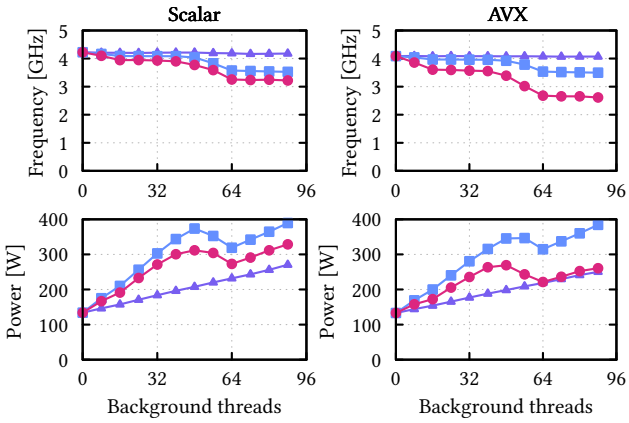


Figure 5: Foreground core frequency (top) and package power (bottom) as cores are assigned to background work (`Matmul`, `Busy`, or `Pause`). `Pause` polling preserves foreground frequency and uses less package power than busy polling.

L1: Idleness transfers power and thermal headroom. An idle service core does not necessarily waste compute. Efficient waiting returns power and thermal headroom that hardware redistributes to productive work.

3.4 Placement Determines Budget Sharing

We investigate the effect of CPU topology on the fixed budget (Figure 6). Each run uses 16 threads for scalar matrix multiplication and 16 threads for polling. We place polling threads in three topologies: applications share dies while using different cores (Die), share the same socket but use disjoint dies (Socket), or share a physical core with sibling hyperthreads (SMT). For each placement, we measure how `Busy` and `Pause` affect matrix multiplication throughput, frequency, and package power.

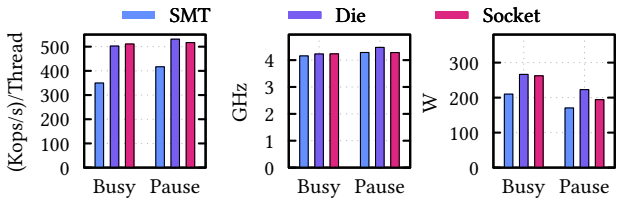


Figure 6: Per-thread scalar matrix multiplication throughput, matrix multiplication core frequency, and package power when co-located on the same CPU with 16 polling threads. Each group compares waiting strategy (`Busy`, `Pause`) with different placement strategies (Die, Socket, SMT).

Waiting strategy reduces interference on SMT cores. When two compute-intensive applications share a physical core, performance degrades when they compete for the same execution units. The correct waiting strategy mitigates the slowdown from polling by freeing the CPU power budget during idle periods. With SMT placement, `Pause` reduces package power by 18.7% compared to `Busy` and improves matrix multiplication throughput by 19.2%.

Thread placement can dissipate hotspots. Individual chiplets can become hotspots, and DVFS can downclock their cores. The right waiting strategy can dissipate these hotspots so applications can run at higher clock frequencies. For example, when `Pause` polling is co-located on the same die as matrix multiplication, the reduced power consumption on that chiplet allows the cores running matrix multiplication to boost from 4.28 GHz with `Busy` polling to 4.47 GHz. This yields a modest 2.8% performance improvement, but the effect is likely to be more pronounced in thermally constrained systems, where localised thermal hotspots more readily limit boost frequency due to less aggressive cooling solutions or sustained high ambient temperatures [25].

L2: Placement decides who shares the budget. The compute budget is distributed hierarchically across cores, chiplets, and sockets. Service placement therefore determines which applications compete for power, frequency, and thermal headroom, not just cache locality.

3.5 Idle Pattern Dictates Waiting Strategy

Figure 7 and Figure 8 show the trade-offs between polling strategies across burst scales (μs, ms, s) and busy:idle ratios (1:9, 5:5, 9:1). We dedicate 96 threads to matrix multiplication, while the remaining threads run a polling application that processes headers and computes a pseudo-checksum during busy periods, similar to network packet processing.

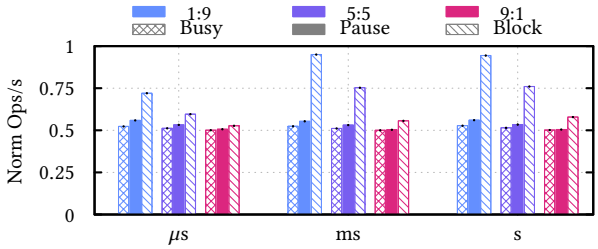


Figure 7: Normalised scalar matrix multiplication throughput with co-located bursty polling workloads using different waiting strategies (`Busy`, `Pause`, and `Block`). Results are grouped by burst timescale (μs, ms, and s) and busy:idle ratios (1:9, 5:5, and 9:1). Blocking frees the most application capacity when idle periods are long enough to amortise scheduler overhead, but `Pause` helps application capacity at short timescales.

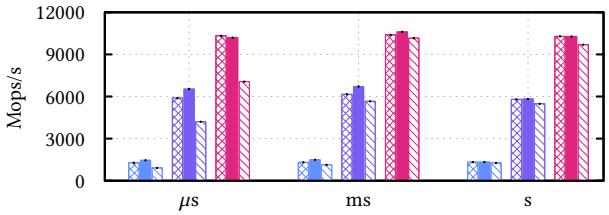


Figure 8: Throughput of a bursty polling application using different waiting strategies, co-located with matrix multiplication. Results are grouped by burst timescale and busy:idle ratios. Blocking loses polling throughput at the microsecond timescale but becomes competitive at longer timescales. `Pause` achieves performance comparable to `Busy` at small timescales.

We report both applications' aggregate throughput for each strategy. With `Block`, polling threads run with real-time priority and share logical cores with additional matrix multiplication threads. Sleeping polling threads yield to matrix multiplication, and waking polling threads immediately preempt them. Figure 7 normalises throughput to matrix multiplication running alone on all 192 threads.

Waiting strategy impacts application performance. Matrix multiplication benefits from blocking polling during idle periods. At the second timescale, blocking reduces throughput by only 5.7% from the baseline when the polling application is busy for one second and idle for nine. At the microsecond timescale, the overheads are not amortised and blocking incurs a 28.1% slowdown relative to the baseline. In comparison, we measure a 47.7% slowdown for busy polling at the microsecond timescale. `Pause` polling does slightly better, at 44.2% slowdown.

Wakeup overheads impact polling performance. The overheads and wakeup latencies of the different strategies influence polling performance. At the microsecond timescale, blocking processes 0.9 million pseudo-packets per second for a 1:9 busy:idle ratio. Polling with `Pause`, in contrast, processes 1.46 million pseudo-packets per second, a 62.2% speedup. At the second timescale, scheduler overheads are amortised and blocking achieves performance comparable to busy and `Pause` polling.

L3: Waiting policy should be adaptive. Waiting spans a spectrum between busy polling and blocking. Different points trade wakeup latency for returned compute budget. Network stacks should adapt their waiting mechanism to expected idle duration instead of treating polling and blocking as fixed alternatives.

L4: Avoid complexity at small timescales. Core reallocation is worthwhile only after scheduler overheads, migration costs, and cache disruption can be amortised. At microsecond timescales, retaining ownership and selecting a better waiting policy to redistribute the fixed budget is often more effective than moving work between cores.

3.6 Downclocking Network Stack Cores Boosts RPC Throughput

The preceding microbenchmarks show how waiting policies and thread placement affect the budget available to productive work. We now test whether the same model explains end-to-end RPC performance with a real service-core network stack. We also examine a second way to redistribute the budget: explicitly reducing network-core frequency, rather than changing how those cores wait.

Figure 9 illustrates this on two AWS `c8a.metal-24xl` bare-metal instances, each with 96 cores (AMD EPYC 9R45). Each host runs Chamelio [35] as its service-core network stack with 48 fast-path cores, and an RPC server application on 48 separate cores. These bare-metal AWS instances are capped at 320 W, so the effects observed here reflect redistribution of power headroom. The client sends fixed 64-byte UDP RPC requests at a constant configured rate.

These AWS instances limit software frequency control to two groups of 48 cores. We therefore place the network datapath and application workers in separate groups to vary datapath frequency independently. This restriction comes from the AWS host control interface; AMD documents per-core boost limits through HSMP, which is unavailable here.

Slowing down network cores increases throughput. In the graph to the left, the server performs 20,000 arithmetic operations per request to model a compute-intensive service. Reducing the server's fast-path frequency from 4.5 to 2.12 GHz increases throughput from 1.60 to 1.84 million requests per second, a 14.6% improvement. Throughput remains near 1.60 million requests per second between 3.71 and 4.5 GHz, then rises as fast-path frequency decreases further. The management unit downclocks the whole package when the CPU hits its power cap. Selectively reducing the frequency of the networking stack gives power headroom to application cores and prevents them from being excessively downclocked. This result is consistent with the fixed-budget model: reducing the fast path's demand for the processor's budget leaves more budget for application workers. The throughput measurements show that maximising fast-path frequency can reduce end-to-end performance.

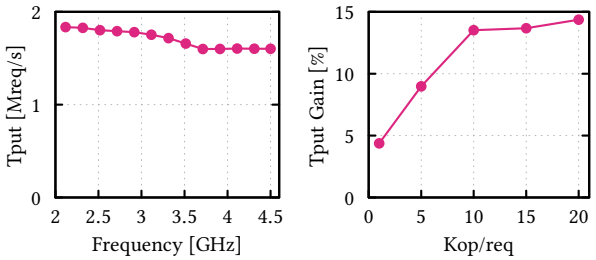


Figure 9: RPC throughput vs. network stack service core frequency (left) and gains from reducing service core frequency with varying work per request (right). Slowing the network stack releases package budget to application cores and improves throughput. More application work per request increases the benefit.

Throughput gains grow with operations per request. In the graph to the right, we vary the amount of work per request. We measure the throughput gain from running Chamelio's fast-path cores at 2.12 GHz instead of 4.5 GHz. The gain from reducing fast-path frequency grows with server work per request. At 1 Kop/req, lower frequencies improve throughput by 4.4%. At 5 Kop/req, the improvement is 9.0%. It reaches 13.5% at 10 Kop/req and about 14% at 15 and 20 Kop/req. As work per request grows, the application benefits more from the headroom released by the network stack.

These results extend the fixed-budget model to a real network stack: reducing the service cores' budget consumption can improve end-to-end throughput. Explicit frequency control therefore provides another way to return headroom to application cores while retaining the service-core allocation.

4 Outlook and Discussion

The fixed-budget model makes managing power and thermal headroom a central concern for service-core systems. We now consider how this perspective should shape network-stack policies and hardware interfaces, and where its benefits depend on workload and platform constraints.

Toward budget-aware network stacks. Service-core designs should decide how much power and thermal budget networking should consume while meeting latency objectives. These decisions become more consequential as core counts grow faster than package power. Waiting policy, placement, and efficient core reallocation [10, 17, 32] can balance application throughput against network responsiveness, with reallocation particularly useful for longer-term workload or locality changes. On chiplet processors, these policies must account for headroom at both die and package levels. A key challenge is deciding when to retain, idle, slow, or reallocate service cores as workload demand and package headroom change. This reasoning extends to other systems, such as NFV, where polling and compute-intensive applications can compete for package headroom even on separate cores.

Power-management interactions. Established power-management techniques determine when and how our findings apply. A tighter package power cap can induce contention at lower concurrency; relaxing it can increase the benefit of additional active cores if thermal headroom permits. Selective DVFS can release headroom while preserving core ownership, as our RPC experiment illustrates (Figure 9), but excessive frequency reduction can make networking the bottleneck. Idle-state selection similarly trades power savings against wakeup latency. Released headroom improves throughput only if productive cores can use it; frequency ceilings or memory bottlenecks can limit the gain. Reallocation should be evaluated against an efficiently waiting, power-managed baseline under the same power cap and service-latency target. Comparisons should report application throughput and service latency.

The interface gap. Hardware controls vary by platform. On our EPYC 9655, the interfaces we tested did not let us explicitly clock selected busy cores lower than others, although a single runtime parameter sufficed on our older AMD and Intel systems. Our AWS experiment instead uses frequency control over groups of 48 cores. Such differences constrain software's allocation of the shared budget.

Waiting poses a related challenge. Network stacks largely rely on busy polling that retains the core, or blocking that relinquishes it to the scheduler. Modern processors provide intermediate hardware mechanisms that return increasing amounts of compute budget while preserving ownership of the core, but these are coarse, difficult to use from userspace and lack adequate portability. Service-core network stacks would benefit from a standard "halt-in-place" primitive that separates waiting from core ownership. It should let a service wait for a queue update or deadline while retaining its core, with a predictable wakeup cost.

Limitations. Our conclusions apply to processors operating near their package power and thermal limits. Outside this regime, the classical model of independent cores remains appropriate. Our evaluation covers recent AMD platforms and focuses on throughput. Evaluating other processor families, designing an adaptive policy, and validating it against tail-latency targets under changing load remain future work.

5 Related Work

Waiting while retaining ownership. Prior work analysed spinning versus blocking for locks [5, 18], and the energy-throughput trade-offs among spinning, `PAUSE`, `MWAIT`, and blocking [8]. Others reduce blocking and wakeup costs [14], exploit `MWAIT` for idle cores [4], or examine how virtualization hides hardware idleness [37]. We examine how these waiting choices affect the benefit of reclaiming idle service cores under a shared power budget.

Trading frequency for latency and energy. A large body of work adapts frequency to workload needs, from sub-request DVFS for latency-critical services [13, 19] to application-controlled scaling [6, 12, 30]. Other work uses feedback to control C-states [39]. The TURBO Diaries [36] explicitly studies slowing or halting some cores to let others boost. Race-to-idle designs such as PowerNap [29] pursue energy proportionality [3]. Our model shows that frequency reduction improves throughput without core reallocation.

Reclaiming and harvesting idle capacity. Microsecond schedulers reclaim idle service cores by reallocation [10, 17, 32], supported by low-overhead handoffs [2, 16, 24] and policies for microsecond tasks [28]. We qualify their shared premise that an idle core represents an independent unit of compute. At the power cap, a reclaimed core runs at reduced frequency, so recovered work is worth less than its core count implies. An efficiently waiting core can already return much of its budget without a software handoff. Reallocation is worthwhile for long idle periods or below the power cap, but not by default for workloads with microsecond idle periods under a binding package power limit.

6 Conclusion

Modern processors increasingly behave as a shared compute budget rather than a collection of independent cores. In this regime, idle service cores are not necessarily wasted, and reclaiming them at microsecond timescales can add complexity without recovering meaningful compute. We hope this perspective guides budget-aware network stacks on large multicore systems.

Acknowledgements

We thank the anonymous reviewers and our shepherd, Gianni Antichi, for their helpful feedback. This work is supported in part by Deutsche Forschungsgemeinschaft (DFG) — 565608348, NSF grant 2450086, and the University of Washington Center for the Future of Cloud Infrastructure (FOCI).

References

[1] Advanced Micro Devices, Inc. AMD EPYC 9655 processor. https://www.amd.com/en/products/processors/server/epyc/9005-series/amd-epyc-9655.html.

[2] Berk Aydogmus, Linsong Guo, Danial Zuberi, Tal Garfinkel, Dean Tullsen, Amy Ousterhout, and Kazem Taram. Extended user interrupts (xUI): Fast and flexible notification without polling. In *30th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS, 2025.

[3] Luiz André Barroso and Urs Hölzle. The case for energy-proportional computing. *Computer*, 40(12), December 2007.

[4] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhania. The Multikernel: A new OS architecture for scalable multicore systems. In *22nd ACM Symposium on Operating Systems Principles*, SOSP, 2009.

[5] Leonid B. Boguslavsky, Karim Harzallah, Alexander Y. Kreinin, Kenneth C. Sevcik, and Alexander Vainshtein. Optimal strategies for spinning and blocking. *Journal of Parallel and Distributed Computing*, 1994.

[6] DPDK Authors. Data plane development kit documentation. 58. power management. https://doc.dpdk.org/guides-24.07/prog_guide/power_man.html.

[7] Hadi Esmaeilzadeh, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger. Dark silicon and the end of multicore scaling. In *38th Annual International Symposium on Computer Architecture*, ISCA, 2011.

[8] Babak Falsafi, Rachid Guerraoui, Javier Picorel, and Vasileios Trigonakis. Unlocking energy. In *2016 USENIX Annual Technical Conference*, ATC, 2016.

[9] Xiaobo Fan, Wolf-Dietrich Weber, and Luiz André Barroso. Power provisioning for a warehouse-sized computer. In *34th Annual International Symposium on Computer Architecture*, ISCA, pages 13–23, 2007.

[10] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation*, OSDI, 2020.

[11] Nikos Hardavellas, Michael Ferdman, Babak Falsafi, and Anastasia Ailamaki. Toward dark silicon in servers. *IEEE Micro*, 31(4):6–15, July 2011.

[12] Tomas Hruby, Herbert Bos, and Andrew S. Tanenbaum. When slower is faster: On heterogeneous multicores for reliable systems. In *2013 USENIX Annual Technical Conference*, ATC, 2013.

[13] Chang-Hong Hsu, Yunqi Zhang, Michael A. Laurenzano, David Meisner, Thomas Wenisch, Ronald G. Dreslinski, Jason Mars, and Lingjia Tang. Reining in long tails in warehouse-scale computers with quick voltage boosting using Adrenaline. *ACM Transactions on Computer Systems*, 35(1), March 2017.

[14] Jack Humphries, Kostis Kaffes, David Mazières, and Christos Kozyrakis. A case against (most) context switches. In *18th Workshop on Hot Topics in Operating Systems*, HOTOS, 2021.

[15] Intel Corporation. Intel 64 and IA-32 architectures software developer's manual. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html, July 2024.

[16] Rishabh Iyer, Musa Unal, Marios Kogias, and George Candea. Achieving microsecond-scale tail latency efficiently with approximate optimal scheduling. In *29th ACM Symposium on Operating Systems Principles*, SOSP, 2023.

[17] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive scheduling for microsecond-scale tail latency. In *16th USENIX Symposium on Networked Systems Design and Implementation*, NSDI, 2019.

[18] Anna R. Karlin, Kai Li, Mark S. Manasse, and Susan S. Owicki. Empirical studies of competitive spinning for a shared-memory multiprocessor. In *13th ACM Symposium on Operating Systems Principles*, SOSP, 1991.

[19] Harshad Kasture, Davide B. Bartolini, Nathan Beckmann, and Daniel Sanchez. Rubik: fast analytical power management for latency-critical systems. In *Proceedings of the 48th International Symposium on Microarchitecture*, MICRO-48, 2015.

[20] Antoine Kaufmann, Tim Stamler, Simon Peter, Naveen Kr. Sharma, Arvind Krishnamurthy, and Thomas Anderson. TAS: TCP acceleration as an OS service. In *14th ACM European Conference on Computer Systems*, EuroSys, 2019.

[21] Alok Gautam Kumbhare, Reza Azimi, Ioannis Manousakis, Anand Bonde, Felipe Frujeri, Nithish Mahalingam, Pulkit A. Misra, Seyyed Ahmad Javadi, Bianca Schroeder, Marcus Fontoura, and Ricardo Bianchini. Prediction-Based power oversubscription in cloud platforms. In *2021 USENIX Annual Technical Conference*, ATC, pages 473–487, 2021.

[22] Malte-Christian Kuns, Hannes Tröpgen, and Robert Schöne. An analysis of user-space idle state instructions on x86 processors. In *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering*, ICPE '25, 2025.

[23] Shaohong Li, Xi Wang, Xiao Zhang, Vasileios Kontorinis, Sreekumar Kodakara, David Lo, and Parthasarathy Ranganathan. Thunderbolt: Throughput-Optimized, Quality-of-Service-Aware power capping at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation*, OSDI, pages 1241–1255, 2020.

[24] Jiazhen Lin, Youmin Chen, Shiwei Gao, and Youyou Lu. Fast core scheduling with userspace process abstraction. In *30th ACM Symposium on Operating Systems Principles*, SOSP, 2024.

[25] Rui Lu and Dan Wang. A thermal-aware workload scheduler for high-performance LLM inference in cooling-regulated datacenters. In *HotCarbon '25: Workshop on Sustainable Computer Systems*, HotCarbon, 2025.

[26] Yenai Ma, Leila Delshadtehrani, Cansu Demirkiran, José L. Abellán, and Ajay Joshi. TAP-2.5D: A thermally-aware chiplet placement methodology for 2.5D systems. In *Proceedings of the 2021 Design, Automation & Test in Europe Conference & Exhibition*, DATE, pages 1246–1251, 2021.

[27] Michael Marty, Marc de Kruijf, Jacob Adriaens, Christopher Alfeld, Sean Bauer, Carlo Contavalli, Michael Dalton, Nandita Dukkipati, William C. Evans, Steve Gribble, Nicholas Kidd, Roman Kononov, Gautam Kumar, Carl Mauer, Emily Musick, Lena Olson, Erik Rubow, Michael Ryan, Kevin Springborn, Paul Turner, Valas Valancius, Xi Wang, and Amin Vahdat. Snap: a microkernel approach to host networking. In *27th ACM Symposium on Operating Systems Principles*, SOSP, 2019.

[28] Sarah McClure, Amy Ousterhout, Scott Shenker, and Sylvia Ratnasamy. Efficient scheduling policies for Microsecond-Scale tasks. In *19th USENIX Symposium on Networked Systems Design and Implementation*, NSDI, pages 1–18, 2022.

[29] David Meisner, Brian T. Gold, and Thomas F. Wenisch. PowerNap: eliminating server idle power. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XIV, 2009.

[30] Louis-Marie Nicolas, Salim Mimouni, Philippe Couvée, and Jalil Boukhobza. Characterizing the use of DVFS for HPC I/O optimization: A microbenchmarking approach. In *5th Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*, CHEOPS, 2025.

[31] Zhixiong Niu, Hong Xu, Peng Cheng, Qiang Su, Yongqiang Xiong, Tao Wang, Dongsu Han, and Keith Winstein. NetKernel: Making network stack part of the virtualized infrastructure. In *2020 USENIX Annual Technical Conference*, ATC, 2020.

[32] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *16th USENIX Symposium on Networked Systems Design and Implementation*, NSDI, pages 361–378, 2019.

[33] Giuseppe Romano, Aakrati Jain, Nima Dehmamy, Cheng Chi, and Xin Zhang. DiffChip: Thermally aware chip placement with automatic differentiation. In *Proceedings of the 75th IEEE Electronic Components and Technology Conference*, ECTC, pages 221–226, 2025.

[34] Matheus Stolet, Liam Arzola, Simon Peter, and Antoine Kaufmann. Tail contagion: Sub-microsecond time protection in shared software network datapaths. *arXiv preprint arXiv:2309.14016*, 2026. https://arxiv.org/abs/2309.14016.

[35] Matheus Stolet, Simon Peter, and Antoine Kaufmann. Chamelio: A fast shared cloud network stack for isolated tenant-defined protocols, 2026. https://arxiv.org/abs/2604.22603.

[36] Jons-Tobias Wamhoff, Stephan Diestelhorst, Christof Fetzer, Patrick Marlier, Pascal Felber, and Dave Dice. The TURBO diaries: Application-controlled frequency scaling explained. In *2014 USENIX Annual Technical Conference*, ATC, 2014.

[37] Yun Wang, Xingguo Jia, Ben Luo, Kenan Liu, Shengdong Dai, Jingdong Han, Weihao Chen, Yicheng Gu, Xingzi Yu, Yibin Shen, Jiesheng Wu, Zhengwei Qi, and Haibing Guan. What are you (M)waiting for: The hidden cost of idle in the hyperscale cloud. In *20th USENIX Symposium on Operating Systems Design and Implementation*, OSDI, 2026.

[38] Qiang Wu, Qingyuan Deng, Lakshmi Ganesh, Chang-Hong Hsu, Yun Jin, Sanjeev Kumar, Bin Li, Justin Meza, and Yee Jiun Song. Dynamo: Facebook's data center-wide power management system. In *43rd Annual International Symposium on Computer Architecture*, ISCA, pages 469–480, 2016.

[39] Xin Zhan, Reza Azimi, Svilen Kanev, David Brooks, and Sherief Reda. CARB: A c-state power management arbiter for latency-critical workloads. *IEEE Computer Architecture Letters*, 16(1):6–9, 2017.